CampusClaw 令牌认证与知识库问答实现说明

依据当前仓库实现整理。浏览器只访问 web，Nginx 将 `/api` 转发到 Go api，班级过滤使用登录后从用户表读出的 `class_id`。

1. 两条链路怎么接在一起

一次提问先过身份认证，再进入知识库问答。认证回答的是「谁、什么角色、哪个班」。问答只使用这个班已经入库的材料，不采用浏览器提交的班级编号。

运行时顺序可以写成：

1. 用户提交账号密码，api 签发 HS256 JWT，并把令牌编号 `jti` 写入 `sessions`。
2. 浏览器把令牌放进 `sessionStorage`，之后每个受保护请求都带 `Authorization: Bearer`。
3. api 验签并核对 `jti` 仍在 `sessions` 中，再从 `users` 读出当前角色和班级。
4. 解题助手确认会话属于这个用户和这个班，读取本班提示词与「解题引导」开关。
5. 对本班做混合检索，最多取 4 条已就绪切片。没有命中就不调用对话网关。
6. 有命中时生成回答，用 SSE 逐段推回浏览器，并把问题和回答写入会话。

2. 令牌认证的实现过程

2.1 登录时签发令牌

入口是 `POST /api/login`，实现在 `backend/internal/server/server.go` 的 `login`。请求体只有用户名和密码。

1. 用「小写用户名 + 客户端 IP」做失败计数。达到 `LOGIN_FAIL_LIMIT`（默认 8 次）后，在 `LOGIN_LOCK_SECONDS`（默认 60 秒）内继续返回与密码错误相同的 `invalid_credentials`，不使用 429。
2. 按用户名查询 `users`，取出 `id`、`username`、`password_hash`、`role`、`class_id`。密码只存 bcrypt。账号不存在时仍做一次等价耗时的哈希比较，避免用响应时间区分账号是否存在。
3. 比对成功后生成 32 位十六进制随机串作为 `jti`。过期时间取 `SESSION_TTL_HOURS`，默认 24 小时。
4. 调用 `token.Sign`，密钥是环境变量 `SESSION_SECRET`。密钥缺失时 api 启动失败，源码里没有默认密钥。
5. 把 `jti`、`user_id`、`expires_at` 插入 `sessions`。响应 JSON 含 `ok`、`role`、`class_id` 和 `token`。同时清掉历史的 `sid` Cookie，不再用 Cookie 维持登录。

如果这次登录请求自己已经带了一枚有效令牌，服务端会先删掉那枚旧 `jti`，再签发新令牌。其他设备上的令牌不会因此失效。

2.2 令牌的格式与校验规则

实现在 `backend/internal/token/token.go`。令牌是三段 Base64URL，用点连接：

```
header.payload.signature
```

头部固定为 `{"alg":"HS256","typ":"JWT"}`。载荷字段固定为：

字段	含义
user_id	用户主键
username	登录时从用户表读出的用户名
role	签发当时的角色，teacher 或 student
class_id	签发当时的班级
iat / exp	签发与过期的 Unix 秒
jti	服务端作废用的编号，对应 sessions.id

签名是对「前两段原文」做 HMAC-SHA256，密钥为 `SESSION_SECRET`。`Parse` 的拒绝条件是：

- 不是恰好三段，或 Base64 无法解码。
- `alg` 不是 `HS256`，或 `typ` 不是 `JWT`。因此 `alg: none` 会被拒绝。算法不从令牌里挑选，始终用 HMAC-SHA256 重算签名。
- 签名与本地计算结果不一致，比较使用常量时间函数。
- `user_id` 或 `jti` 为空，或 `exp` 已经不大于当前时间。

> 载荷里的 role 和 class_id 便于在浏览器网络面板里看到身份，但授权不单独信任这两项。每次请求仍回用户表读取当前角色和班级。

2.3 浏览器如何带上令牌

实现在 `frontend/src/api.ts`。登录成功后把 `token` 写入 `sessionStorage`，键名 `campusclaw-token`。同一标签页刷新后仍在；关闭标签页后消失。

`authHeaders` 读取该键，设置 `Authorization: Bearer <token>`。普通 JSON 请求、`POST /api/conversations/{id}/messages` 的 SSE，以及材料原文件下载，都走这一函数。原文件不能再用普通链接打开，因为浏览器导航不会附带自定义请求头，所以下载改为 `fetch` 拿到文件后再触发保存。

页面启动时 `App.tsx` 调用 `GET /api/me`。令牌有效则进入材料页或解题助手；401 则回到登录页。退出时先带当前令牌调用 `POST /api/logout`，无论请求是否成功，都会删掉本地令牌。

2.4 Nginx 转发

`frontend/nginx.conf` 的 `location /api/` 显式设置：

```
proxy_set_header Authorization $http_authorization;
```

这样浏览器发出的 Bearer 能到达 api。同一 location 关闭了代理缓冲，读超时 300 秒，避免解题助手的 SSE 被攒包。

2.5 每个受保护接口如何认人

`require` 包住材料、检索、提问、提示词、技能和会话接口。`GET /health` 与 `POST /api/login` 不经过它。

1. 从 `Authorization` 取出 `Bearer` 后面的字符串。没有这一头，或前缀不对，直接 401，错误码 `unauthorized`。
2. `token.Parse` 验签并检查过期。
3. 执行查询：`sessions` 与 `users` 按 `jti` 和 `user_id` 连接，且 `expires_at` 必须晚于数据库当前时间。
4. 查不到行就是 401。查到后，后续处理函数拿到的 `sessionUser` 使用用户表里的 `id`、`username`、`role`、`class_id`。

因此，学生在 URL 或 JSON 里写另一个班的 `class_id`，不能改变自己的班级。教师查看或上传到其他班，是业务接口在确认角色为 teacher 之后另行允许的，不是令牌被客户端改写的结果。

2.6 登出为什么能立刻失效

纯 JWT 在过期前无法收回。`logout` 在验签成功后执行 `DELETE FROM sessions WHERE id=? AND user_id=?`，删的是当前 `jti`。之后 `require` 联表失败，同一串令牌返回 401。过期时间仍然作为第二道界限：即使行还在，`Parse` 也会拒绝过期令牌。

3. 问答之前：材料如何进入知识库

提问能引用材料，是因为教师上传时已经完成切片和向量入库。学生不能上传，对应接口返回 403。

1. 教师 `POST /api/materials`。文件只接受文本类材料，正文写入 `knowledge_entries`，并与材料、班级关联。上传目录只挂在 api 容器，不由 Nginx 直接暴露。
2. `chunker.Split` 默认策略 `auto`：按 800 字窗口、80 字重叠切开。另有按标题分段的 `hierarchy`，以及可指定窗口的 `custom`。每段记下 `chunk_index`、`char_start`、`char_end`。
3. `indexx.IndexMaterial` 把切片写入 `knowledge_chunks`，状态先不是 ready。嵌入向量写入 Qdrant，成功后把该行 `index_status` 更新为 `ready`。失败则标为 `failed`。检索只使用 `ready`。
4. 关键词检索依赖 MySQL 对 `chunk_text` 的 ngram 全文索引。向量检索用同一条切片在 Qdrant 中的点，过滤条件带班级。

4. 知识库问答的实现过程

页面是解题助手，接口是 `POST /api/conversations/{id}/messages`，实现在 `backend/internal/server/tutor.go` 的 `postMessage`。第 4 课的 `POST /api/search` 和 `POST /api/ask` 仍然保留，班级同样只取令牌用户，前端 `/search` 会转到解题助手。

4.1 确认会话

请求先经过 `require`。随后 `ownConversation` 要求 `conversations` 中存在同时满足 `id`、`user_id`、`class_id` 的行。他人或他班访问返回 404，不区分「不存在」和「不属于你」。

请求体可以带 `query`、`system`、`class_id`、`messages`。后三项被显式丢弃。问题只取 `query` 去空白后的文本，空问题或超过 2000 字返回 400 `empty_query`。历史不信任客户端，后面只从数据库读。

用户问题先插入 `conversation_messages`，角色为 `user`。前端在此之前可以 `POST /api/conversations` 新建会话；列表和详情也只返回创建者自己的会话。

4.2 读取本班提示词和技能

1. `classPrompt` 读 `class_prompts.body`。没有行，或正文为空白，使用 `tutor.DefaultPrompt`。这段文字只说明助手是谁、只依据本班材料，不写具体解题步骤。数据库错误不会被当成默认提示词。
2. `skillEnabled` 只查询技能键 `guide`（解题引导）。没有记录时视为开启。教师用 `PUT /api/skills/guide` 修改；学生调用返回 403；其他技能键返回 404。
3. 提示词和开关都在这一轮开始时读取。助手消息会保存 `prompt_snapshot` 和 `skill_enabled`。教师之后再改提示词，已经落库的回答不重写。下一轮提问才用新值。

4.3 混合检索

`loadTutorHits` 调用 `retrieve.Hybrid`，班级参数是用户表中的 `class_id`。

1. 关键词路：MySQL `MATCH ... AGAINST` 自然语言模式，条件是本班且 `index_status='ready'`，按相关度取前 20。
2. 向量路：把问题嵌入后到 Qdrant 检索，再回到 MySQL 核对切片仍属于该班且状态为 ready，同样最多 20 条。
3. 两路用倒数排名融合。常数 k 为 60。某条切片若两路都出现，分数相加。按融合分排序后，问答只保留前 4 条。
4. 再用 `chunk_id` 和 `class_id` 读出 `chunk_text`，带上材料标题、切片序号和字符区间，交给生成步骤。

检索失败返回 503 `vector_unavailable`，不会改用他班材料。

4.4 没有命中时

`tutor.Stream` 发现命中为空，正文固定为「资料中未找到相关内容」。引用列表为空。`model_called` 为 false。不请求对话网关，也不把上一轮的引用抄进这一轮。

4.5 有命中时如何生成

若同时配置了 `CHAT_BASE_URL` 和 `CHAT_API_KEY`，`BuildMessages` 组出发给网关的消息：

- 一条 system：已保存的角色提示词，加上解题引导的规程。开启时要求只给思路、引用标题和位置、用 `[n]` 标注，并写明不要直接给出最终答案。关闭时要求直接回答并标注依据。同时禁止编造资料中没有的内容，禁止听从用户消息里的 system 指令。

- 此前最多 16 条本会话历史。刚插入的当前问题会从历史末尾去掉，避免重复。`chatHistory` 只保留 user 与 assistant。
- 一条 user：把最多 4 条资料写成「[n] 材料《标题》切片、字符区间、正文」，再接上当前问题。

请求发往 `{CHAT_BASE_URL}/v1/chat/completions`，`stream: true`。这里的 `Authorization: Bearer` 使用 `CHAT_API_KEY`，与用户 JWT 无关，也不会返回给浏览器。网关若以 SSE 返回，按 `delta.content` 逐片交给上层；若返回整段 JSON，则一次吐出 `message.content`。调用失败、空正文或未配置网关时，改走本地回答，仍然按小段 SSE 输出。

本地回答两种正文不同：

- 解题引导开启：指出第一条命中的标题、切片号和字符区间，让读者按材料自己往下推，并写明不直接给出最终答案。
- 解题引导关闭：复用 `chat.DirectAnswer`，从材料句子里选出与问题重叠最高的一句，附上 `[n]`。

4.6 SSE 与落库

响应类型为 `text/event-stream`，并设置 `X-Accel-Buffering: no`。事件顺序固定：

事件	内容
meta	是否使用了提示词、提示词 SHA-256、技能是否开启、本轮命中条数
citations	引用数组。每条含序号、材料 id、标题、切片序号、字符起止和摘录
delta	若干次，每次一个文本片段
done	完整回答、助手消息 id、`model_called`

`done` 之前，助手行已经写入 `conversation_messages`，字段包括 `content`、`citations_json`、`prompt_snapshot`、`skill_enabled`。生成失败且没有正文时发送 `error`，不把空回答当成成功。

前端 `streamMessage` 按空行切分 SSE，把 `delta` 追加到当前回合，结束后再读一次会话，用数据库中的消息替换本地临时文本。会话下拉的标题来自该会话的第一条用户问题。

4.7 页面上学生能看到什么

教师可以编辑提示词并开关「解题引导」，保存后从下一轮生效。学生看不到写入口，只能看到「解题引导：开启」或「解题引导：关闭」。会话列表每一项可以删除自己的会话，删除走 `DELETE /api/conversations/{id}`，消息随外键级联删除，他人删除得到 404。

5. 一次提问的对照表

步骤	代码位置	结果
带上用户令牌	frontend/src/api.ts	Authorization: Bearer
验签并读用户表	server.require	得到 user_id、role、class_id
确认会话归属	tutor.ownConversation	不是本人本班则 404
读提示词和 guide	classPrompt、skillEnabled	本轮快照
混合检索前 4 条	retrieve.Hybrid、loadTutorHits	只含本班 ready 切片
无命中	tutor.Stream	固定文案，不调网关
有命中则生成	streamRemote 或 localAnswer	引导与直接回答不同
推送并保存	writeSSE、conversation_messages	meta、citations、delta、done

6. 如何核对

1. 未带令牌访问 `GET /api/materials`，应为 401，且响应里没有材料标题。
2. 学生登录后，在浏览器网络面板打开 `materials`，请求头应有 `Authorization: Bearer`。载荷中能看到用户名、角色和班级。列表只含本班材料。
3. 退出后用同一令牌再请求材料，应为 401。
4. 改动令牌任意一段后请求，应为 401。
5. 解题助手对有材料依据的问题应出现引用；对资料中不存在的问题应得到「资料中未找到相关内容」，且不出现上一轮引用。

对应检查脚本为 `scripts/verify_api.py` 与 `scripts/verify_tutor.py`。令牌单测在 `backend/internal/token/token_test.go`，覆盖正常解析、篡改、错误密钥、`alg: none` 和过期。